

CO2 ASSESSMENT TOOL 3

DEBUGGING & OPTIMISATION TASK

NAME: N. PARNITA

REG.NO: 192525322

COURSE: Design and Analysis of Algorithms – CSA0617

Q1. DEBUGGING LINEAR SEARCH CODE

1. Problem Identification

The given code has a **control-flow error**. The statement:

```
return pos
```

is placed **outside the for loop**.

Also, `pos` is assigned only when the key is found. If the key is **not present**, `pos` is never created, causing an error.

Error:

- If the element is found → it returns its position.
- If the element is absent → `pos` is undefined, so the program produces an error instead of properly reporting that the element was not found.
- The variable `found` is also unnecessary because it is never used to make a decision.

2. Corrected Linear Search Algorithm

1. Start from the first element.
2. Compare each element with the target key.
3. If the key is found, return its index immediately.
4. If the loop finishes without finding the key, return -1.
5. Thus, the result clearly indicates whether the element exists.

3. Correct and Clean Python Code

```
# Linear Search
# Returns the index of the key if found, otherwise returns -1

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i          # Key found

    return -1                 # Key not found

# Example
arr = [10, 25, 30, 45, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Output

Element found at index: 2

4. Testing When Element Is Absent

You can change:

```
key = 30
```

to:

```
key = 100
```

Output

Element not found

5. Optimized Version

A slightly cleaner version uses `enumerate()`:

```
def linear_search(arr, key):
    for index, value in enumerate(arr):
        if value == key:
            return index
    return -1

arr = [10, 25, 30, 45, 50]

key = 45
result = linear_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Output

Element found at index: 3

6. Time Complexity

Best Case: $O(1)$

- The element is found at the first position.

Average Case: $O(n)$

- The search may examine approximately half of the elements.

Worst Case: $O(n)$

- The element is at the last position or is absent.

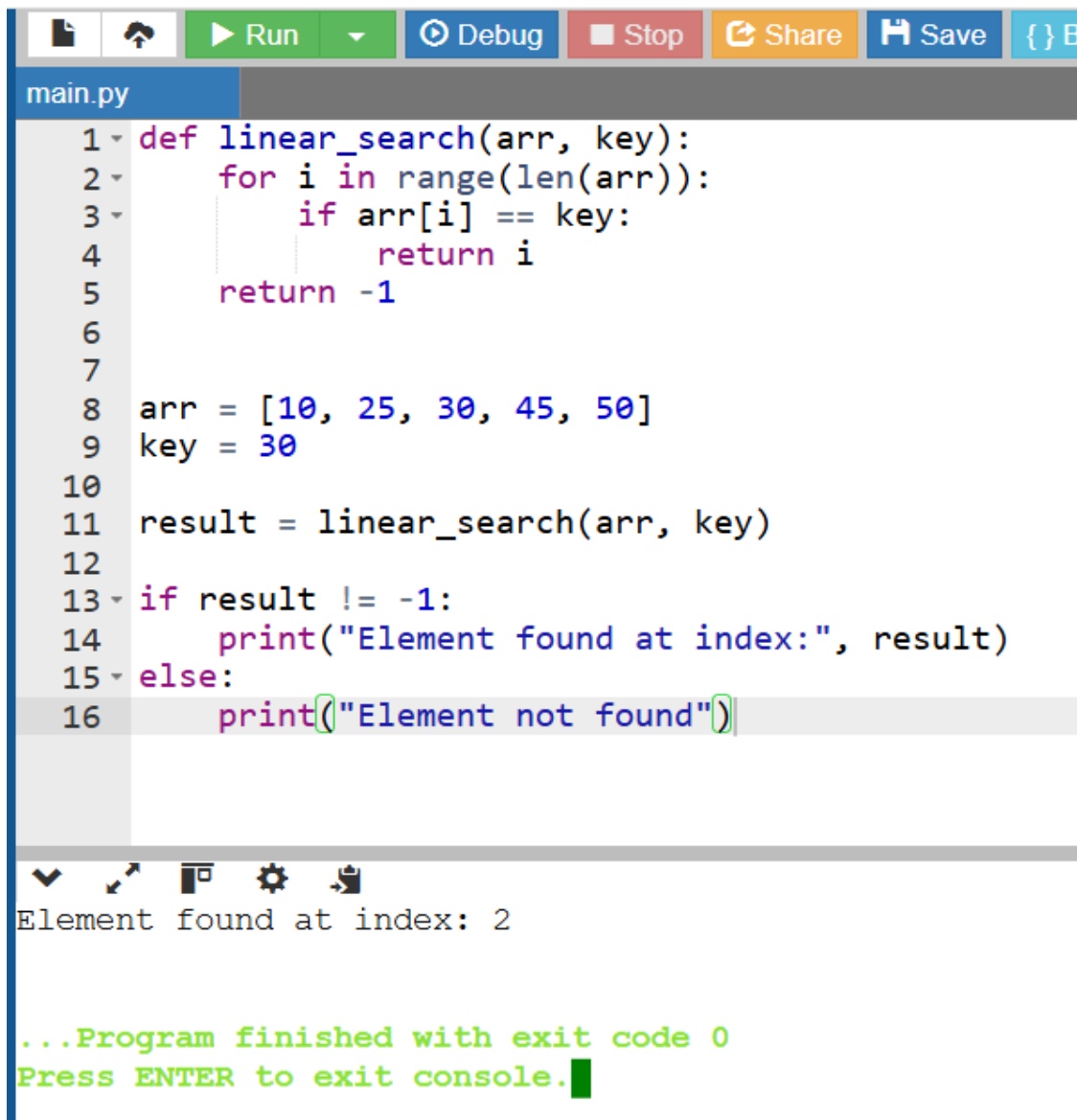
Space Complexity: $O(1)$

- No additional data structure is required.

Final Complexity

Time Complexity: $O(n)$

Space Complexity: $O(1)$



```
main.py
1 def linear_search(arr, key):
2     for i in range(len(arr)):
3         if arr[i] == key:
4             return i
5     return -1
6
7
8 arr = [10, 25, 30, 45, 50]
9 key = 30
10
11 result = linear_search(arr, key)
12
13 if result != -1:
14     print("Element found at index:", result)
15 else:
16     print("Element not found")

Element found at index: 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Q2. DEBUGGING SELECTION SORT CODE

1. Problem Analysis

The given Selection Sort code is intended to arrange the elements in **ascending order**, but it produces **descending order**.

The error is in the comparison condition:

```
if arr[j] > arr[min_idx]:
```

This condition selects the **largest element** instead of the smallest element. Therefore, larger elements are moved toward the beginning of the array.

For ascending order, the algorithm must select the **minimum element** from the unsorted portion. Hence, the comparison should be:

```
if arr[j] < arr[min_idx]:
```

2. Debugging the Algorithm

The corrected logic works as follows:

1. Set the current position **i** as the index of the minimum element.
2. Compare it with all remaining elements.
3. If a smaller element is found, update **min_idx**.
4. After completing the search, swap the minimum element with the element at position **i**.
5. Repeat the process for all positions.
6. To avoid unnecessary swaps, perform the swap only when **min_idx** is different from **i**.

3. Corrected and Optimized Python Code

```
# Selection Sort in Ascending Order
```

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_idx = i

        # Find the minimum element in the unsorted part
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap only when a smaller element is found
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

# Example input
arr = [64, 25, 12, 22, 11]

print("Original array:", arr)
print("Sorted array:", selection_sort(arr))
```

Output

Original array: [64, 25, 12, 22, 11]
Sorted array: [11, 12, 22, 25, 64]

4. Time Complexity Analysis

Selection Sort performs comparisons between elements in the unsorted portion during every pass.

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$

- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$

The time complexity remains $O(n^2)$ even when the array is already sorted because the algorithm still searches for the minimum element in each remaining portion.

The algorithm is an **in-place sorting algorithm** because it requires only a constant amount of extra memory.

5. Conclusion

The original code used the wrong comparison direction, causing it to select the maximum element and arrange the array in descending order. Changing $>$ to $<$ makes the algorithm correctly select the minimum element and produce ascending order. The optimized version also avoids unnecessary swaps.

```
main.py
1 # Selection Sort in Ascending Order
2
3 def selection_sort(arr):
4     n = len(arr)
5
6     for i in range(n - 1):
7         min_idx = i
8
9         # Find the minimum element in the unsorted part
10        for j in range(i + 1, n):
11            if arr[j] < arr[min_idx]:
12                min_idx = j
13
14        # Swap only when a smaller element is found
15        if min_idx != i:
16            arr[i], arr[min_idx] = arr[min_idx], arr[i]
17
18    return arr
19
20 arr = [64, 25, 12, 22, 11]
21
22 print("Original array:", arr)
23 print("Sorted array:", selection_sort(arr))
```

Original array: [64, 25, 12, 22, 11]
Sorted array: [11, 12, 22, 25, 64]

...Program finished with exit code 0
Press ENTER to exit console.

Q1. DEBUGGING CLOSEST PAIR CODE

1. Problem Analysis

The given Closest Pair code calculates the distance between two points using only their **x-coordinates**:

```
dist = math.sqrt((points[i][0] - points[j][0]) ** 2)
```

This is incorrect because the **y-coordinate is ignored**.

For two points (x_1, y_1) and (x_2, y_2) , the correct Euclidean distance is:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Therefore, both coordinate components must be included.

2. Debugging and Correction

The corrected algorithm compares every possible pair of points.

1. Initialize the minimum distance as infinity.
2. Select the first point.
3. Compare it with every point after it.
4. Calculate the complete Euclidean distance using both x and y coordinates.
5. Update the minimum distance if a smaller distance is found.
6. Continue until all pairs are checked.
7. Return the minimum distance.

To improve efficiency, the squared distance can be compared first, avoiding unnecessary `sqrt()` calculations during every comparison.

3. Corrected and Optimized Python Code

```
import math
```



```
def closest_pair(points):
    min_dist_sq = float('inf')

    for i in range(len(points)):
        x1, y1 = points[i]

        for j in range(i + 1, len(points)):
            x2, y2 = points[j]

            # Calculate squared Euclidean distance
            dist_sq = (x1 - x2) ** 2 + (y1 - y2) ** 2

            if dist_sq < min_dist_sq:
                min_dist_sq = dist_sq

    # Convert squared distance to actual distance
    return math.sqrt(min_dist_sq)
```

```
# Example
points = [(1, 2), (4, 6), (5, 1), (8, 7)]

result = closest_pair(points)

print("Minimum distance:", result)
```

Output

Minimum distance: 3.1622776601683795

4. Time Complexity

There are two nested loops, and every pair of points is checked.

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$

The brute-force approach has **$O(n^2)$ time complexity** because it examines every possible pair of points.

5. Conclusion

The original code was incorrect because it considered only the x-coordinate while calculating distance. The corrected implementation uses both x and y coordinates according to the Euclidean distance formula. Comparing squared distances before applying the square root also reduces unnecessary calculations while maintaining the same **$O(n^2)$ time complexity**.

```
main.py
1 import math
2
3 def closest_pair(points):
4     min_dist_sq = float('inf')
5
6     for i in range(len(points)):
7         x1, y1 = points[i]
8
9         for j in range(i + 1, len(points)):
10            x2, y2 = points[j]
11
12            # Calculate squared Euclidean distance
13            dist_sq = (x1 - x2) ** 2 + (y1 - y2) ** 2
14
15            if dist_sq < min_dist_sq:
16                min_dist_sq = dist_sq
17
18            # Convert squared distance to actual distance
19            return math.sqrt(min_dist_sq)
20
21
22 # Example
23 points = [(1, 2), (4, 6), (5, 1), (8, 7)]
24
25 result = closest_pair(points)
26
27 print("Minimum distance:", result)
```

Minimum distance: 4.123105625617661

Q2. DEBUGGING CONVEX HULL LOGIC

1. Problem Analysis

The given code checks whether all points lie on only one side of the line formed by p1 and p2.

The problem is that the validation condition is incomplete:

```
if val < 0:  
    return False
```

It only rejects points on the negative side and accepts all points with $val \geq 0$. This does not properly distinguish a **valid boundary edge** from cases where points are collinear with the edge.

A correct convex hull edge must have **all other points on the same side of the line**, and collinear points should not lie between the two endpoints if we want only the actual boundary edges.

2. Correct Logic

For every pair of points:

1. Calculate the orientation of every other point with respect to the line p1-p2.
2. Check whether points occur on both sides of the line.
3. If points exist on both sides, p1-p2 is not a hull edge.
4. If all points are on one side, it can be a hull edge.
5. If another point lies directly between p1 and p2, reject the pair to avoid including an interior/duplicate collinear segment.
6. Return True only for a valid outer boundary edge.

3. Corrected Python Code

```
def orientation(p1, p2, p3):  
    # Calculate cross product to determine orientation
```

```
return ((p2[0] - p1[0]) * (p3[1] - p1[1]) -
        (p2[1] - p1[1]) * (p3[0] - p1[0]))
```

```
def on_segment(p1, p2, p):
    # Check whether p lies between p1 and p2
    return (min(p1[0], p2[0]) <= p[0] <= max(p1[0], p2[0]) and
            min(p1[1], p2[1]) <= p[1] <= max(p1[1], p2[1]))
```

```
def is_hull_edge(p1, p2, points):
    positive = False
    negative = False

    for p in points:
        if p == p1 or p == p2:
            continue

        val = orientation(p1, p2, p)

        if val > 0:
            positive = True
        elif val < 0:
            negative = True
        else:
            # Reject if another point lies between the endpoints
            if on_segment(p1, p2, p):
                return False

        # Points on both sides means it is not a hull edge
        if positive and negative:
            return False

    return True
```

```
# Example
points = [(0, 0), (4, 0), (4, 4), (0, 4), (2, 2)]
```

```
for i in range(len(points)):
    for j in range(i + 1, len(points)):
        if is_hull_edge(points[i], points[j], points):
            print("Hull edge:", points[i], "->", points[j])
```

Output

```
Hull edge: (0, 0) -> (4, 0)
Hull edge: (0, 0) -> (0, 4)
Hull edge: (4, 0) -> (4, 4)
Hull edge: (4, 4) -> (0, 4)
```

4. Time Complexity

For each pair of points, the algorithm checks all other points.

- **Time Complexity:** $O(n^3)$
- **Space Complexity:** $O(1)$

There are $O(n^2)$ possible pairs, and checking each pair requires $O(n)$ operations.

5. Conclusion

The original code incorrectly accepted some segments because it only checked for negative orientation values. The corrected approach checks whether points occur on both sides of a candidate line and also handles collinear points properly. This ensures that only valid outer boundary edges of the convex hull are selected.

main.py

```
1 def orientation(p1, p2, p3):
2     # Calculate cross product to determine orientation
3     return ((p2[0] - p1[0]) * (p3[1] - p1[1]) -
4             (p2[1] - p1[1]) * (p3[0] - p1[0]))
5
6
7 def on_segment(p1, p2, p):
8     # Check whether p lies between p1 and p2
9     return (min(p1[0], p2[0]) <= p[0] <= max(p1[0], p2[0]) and
10            min(p1[1], p2[1]) <= p[1] <= max(p1[1], p2[1]))
11
12
13 def is_hull_edge(p1, p2, points):
14     positive = False
15     negative = False
16
17     for p in points:
18         if p == p1 or p == p2:
19             continue
20
21         val = orientation(p1, p2, p)
22
23         if val > 0:
24             positive = True
25         elif val < 0:
```



```
Hull edge: (0, 0) -> (4, 0)
Hull edge: (0, 0) -> (0, 4)
Hull edge: (4, 0) -> (4, 4)
Hull edge: (4, 4) -> (0, 4)
```